Information Retrieval

Complete Notes with Detailed Code Explanation

Lab	Topic	Main tools
IR-1	Text pre-processing (cleaning, tokenization, stop words, stemming)	re, NLTK
IR-2	Inverted index and Boolean (AND) search	Pure Python
IR-3	Bayesian Network (probabilistic reasoning)	pandas, numpy, pgmpy
IR-4	PageRank (link analysis)	NetworkX
IR-5	Web scraping / crawling	requests, BeautifulSoup, pandas

Contents: Part 1 - Foundations of IR, history and inventors | Part 2 to 6 - one chapter per notebook (theory, line-by-line code explanation, output explanation, limitations, viva questions) | Part 7 - formulas, comparison table, glossary, revision sheet.

Part 1 - Foundations of Information Retrieval

1.1 What is Information Retrieval (IR)?

Information Retrieval is the science and engineering of finding material (usually documents) of an *unstructured* nature (usually text) that satisfies an *information need* from within large collections (usually stored on computers). The standard textbook definition (Manning, Raghavan and Schutze) is exactly this. When you type a query into Google, search your mail, or look for a product on an e-commerce site, you are using an IR system.

Three words matter in the definition:

- **Unstructured** - text has no fixed rows and columns like a database table. A web page, a PDF or a tweet has no schema, so we cannot simply run SQL on it.
- **Information need** - what the user really wants to know. The **query** is only an imperfect expression of it (e.g. the need is 'how to cure a headache without tablets', the query is 'headache home remedy').
- **Large collection (corpus)** - millions or billions of documents, so the system must be fast and must *rank* results by relevance instead of just listing them.

1.2 IR versus Database retrieval versus Data mining

Aspect	Database (DBMS)	Information Retrieval
Data	Structured (tables, fixed schema)	Unstructured / semi-structured text
Query	Exact and formal (SQL)	Vague, natural language, keywords
Matching	Exact match - a row matches or does not	Partial match - documents are scored by degree of relevance
Result	Set of rows, no order needed	Ranked list (best first)
Errors	Result is either correct or wrong	Relevance is subjective; evaluated with precision and recall
Example	SELECT * FROM students WHERE id=5	'best budget phones 2026'

1.3 Who invented Information Retrieval? (History and inventors)

No single person invented IR. It grew over decades as librarians, mathematicians, computer scientists and later web engineers added ideas. The table below lists the key contributors in chronological order - this is the part examiners usually ask.

Year	Person / group	Contribution
1945	Vannevar Bush	Essay 'As We May Think' describing the **Memex**, a machine to store and link all of a person's books and records. It is considered the intellectual ancestor of IR and hypertext.
1950	Calvin Mooers	**Coined the term 'Information Retrieval'** and worked on descriptor-based (Zatocoding) retrieval. So the name itself is credited to Mooers.
1950s	Hans Peter Luhn (IBM)	Pioneer of automatic indexing. Proposed using **word frequency** to measure significance of words, built KWIC (Key Word In Context) index. Often called the 'father of IR' / 'father of business intelligence'.

Year	Person / group	Contribution
1958-60	Cyril Cleverdon (Cranfield, UK)	**Cranfield experiments** - created the test-collection method and the measures **precision and recall**, still the base of IR evaluation.
1960	Maron and Kuhns	Proposed **probabilistic indexing** - first probabilistic view of retrieval.
1960s-70s	Gerard Salton (Harvard, then Cornell)	Built the **SMART** system, invented the **Vector Space Model**, TF-IDF weighting, relevance feedback and cosine similarity ranking. Widely called the '**father of modern IR**'.
1972	Karen Sparck Jones	Invented **Inverse Document Frequency (IDF)** - the 'I' in TF-IDF. Rare words are more informative than common words.
1975	C. J. van Rijsbergen	Book 'Information Retrieval'; formalised probabilistic and logical models; the F-measure comes from his work.
1976 onwards	Stephen Robertson, Sparck Jones	**Probabilistic Relevance Framework** and later the **BM25** (Okapi) ranking function, still used inside Elasticsearch and Lucene.
1980	Martin Porter	**Porter Stemming Algorithm** (used in IR-1).
1980s	Judea Pearl	Invented **Bayesian Networks** (used in IR-3). Won the Turing Award in 2011.
1990	Alan Emtage	**Archie** - first Internet search tool (searched FTP file names).
1990	Turtle and Croft	**Inference Network / Bayesian network model** of IR (INQUERY system) - links IR with Pearl's Bayesian networks.
1992	NIST / DARPA	**TREC** (Text REtrieval Conference) started - the big annual evaluation benchmark.
1993-94	Matthew Gray; Brian Pinkerton	**World Wide Web Wanderer** (first web crawler, 1993) and **WebCrawler** (1994, first full-text web search engine).
1998	Larry Page and Sergey Brin (Stanford)	**PageRank** (used in IR-4) and the Google search engine.
1998	Jon Kleinberg	**HITS** algorithm (hubs and authorities), an alternative link-analysis method.
1999	Doug Cutting	**Apache Lucene** library - base of Solr and Elasticsearch.
2013 onwards	Mikolov et al., Google, OpenAI etc.	Word embeddings, then BERT (2018) and **dense / neural retrieval**, and later retrieval-augmented generation (RAG) with LLMs.

Exam answer: If asked 'who invented IR?' answer: the **term** was coined by **Calvin Mooers (1950)**; **Gerard Salton** is regarded as the father of modern IR; **Hans Peter Luhn** as the pioneer of automatic indexing; and **Cyril Cleverdon** as the founder of IR evaluation. PageRank was invented by **Larry Page and Sergey Brin**.

1.4 Architecture of an IR system

An IR system has two parallel pipelines that meet at the index.

Stage	What happens	Covered in
1. Crawling / acquisition	Collect documents (web pages, files) from the source.	IR-5
2. Text pre-processing	Clean, tokenize, remove stop words, stem/lemmatize.	IR-1
3. Indexing	Build the inverted index (term -> list of documents).	IR-2
4. Query processing	Apply the same pre-processing to the user's query.	IR-1, IR-2
5. Matching and ranking	Score documents - TF-IDF, BM25, probabilistic models, link analysis (PageRank).	IR-3, IR-4
6. Presentation and feedback	Show results, snippets; learn from clicks and relevance feedback.	-

So the five notebooks are not random programs - together they walk through the **complete IR pipeline**: get the data (IR-5) -> clean it (IR-1) -> index it (IR-2) -> rank it using probability (IR-3) and link structure (IR-4).

1.5 Retrieval models (how documents are matched and ranked)

(a) Boolean model

Documents and queries are sets of terms; queries use AND, OR, NOT. A document either matches or does not - no ranking. Fast and precise but the user must know exactly what to ask. The search function in IR-2 is a Boolean AND model.

(b) Vector Space Model (Salton)

Every document and query is a vector in a space whose dimensions are vocabulary terms. Each component is a weight, usually TF-IDF. Relevance is the cosine of the angle between query and document vectors:

```
TF-IDF(t,d) = tf(t,d) x log( N / df(t) )
cosine(q,d)  = (q . d) / ( |q| x |d| )
tf = times term t appears in document d,  df = number of documents containing t,  N = total documents
```

(c) Probabilistic model

Ranks documents by the probability that they are relevant to the query: P(R | d, q). Based on the Probability Ranking Principle (Robertson, 1977): if documents are ordered by decreasing probability of relevance, the overall effectiveness is the best possible. BM25 is the most successful descendant. Bayesian networks (IR-3) belong to this family.

```
BM25(q,d) = sum over terms t in q of  IDF(t) x  tf x (k1 + 1) / ( tf + k1 x (1 - b + b x |d| / avgdl) )
typical values: k1 = 1.2 to 2.0,  b = 0.75
```

(d) Language models and (e) Neural / dense retrieval

Language-model retrieval ranks documents by how likely the document's language model is to generate the query. Neural retrieval (BERT, sentence embeddings, vector databases) represents text as dense vectors capturing meaning, so 'car' can match 'automobile'. Modern systems combine BM25 (lexical) with dense retrieval (semantic) - 'hybrid search'.

(f) Link-analysis models

Use the hyperlink graph of the web. A page linked by many important pages is itself important. PageRank (IR-4) and HITS are the standard examples.

1.6 Evaluation of IR systems

Because relevance is subjective, IR is evaluated using test collections: a set of documents, a set of queries, and human relevance judgements.

```
Precision = relevant retrieved / total retrieved          (how clean are the results?)
Recall    = relevant retrieved / total relevant in corpus (how complete are the results?)
F1        = 2 x P x R / (P + R)
AP (Average Precision) = mean of precision at each rank where a relevant doc appears;  MAP = mean of AP over all queries
nDCG = discounted cumulative gain normalised - rewards relevant docs placed higher in the list
```

Example: corpus has 20 relevant documents; system returns 10 documents of which 8 are relevant. Precision = 8/10 = 0.8; Recall = 8/20 = 0.4; F1 = 2(0.8)(0.4)/(1.2) = 0.533.

1.7 Applications of IR

- Web search engines (Google, Bing), enterprise and desktop search

- Digital libraries, legal and patent search, medical literature search (PubMed)
- E-commerce product search and recommender systems
- Question answering, chatbots and Retrieval-Augmented Generation (RAG) for LLMs
- Email spam filtering and text classification
- Plagiarism detection, news aggregation, social-media monitoring

Part 2 - Lab IR-1: Text Pre-processing

2.1 Why pre-processing?

Raw text is noisy: capital letters, punctuation, filler words ('the', 'is') and many word forms ('retrieve', 'retrieval', 'retrieving'). A computer would treat each of these as a different token. Pre-processing converts raw text into a **normalised list of index terms**, so that a query word and a document word that mean the same thing actually match. It also shrinks the index and removes meaningless matches. The **same** pre-processing must be applied to documents (at indexing time) and to queries (at search time), otherwise matching fails.

2.2 Pipeline steps - theory

Step 1: Case folding and cleaning

Convert to lowercase so 'Search' and 'search' are one term. Remove punctuation and numbers (noise for simple IR). Caution: in real systems case can matter ('US' vs 'us', 'Apple' vs 'apple'), and numbers can matter (model numbers, dates, prices).

Step 2: Tokenization

Splitting the character stream into **tokens** (words). It sounds trivial but has traps: 'don't' (one or two tokens?), 'New York' (one entity), hyphens ('pre-processing'), e-mail addresses, URLs, and languages without spaces (Chinese, Japanese). NLTK's `word_tokenize` uses the Punkt sentence splitter plus the Penn Treebank word tokenizer rules.

Step 3: Stop-word removal

Stop words are extremely frequent function words (the, is, of, and, to...) that carry little meaning. Removing them reduces index size by roughly 25-30 percent and speeds search (Luhn's observation, and Zipf's law: a few words account for most of the text). Trade-off: queries such as 'To be or not to be' or 'The Who' break if every stop word is dropped, so modern engines often keep them and rely on compression.

Step 4: Stemming

Stemming **chops word endings** using rules to reach a common root (stem). 'retrieval', 'retrieving', 'retrieves' -> 'retriev'. It is fast and language-rule based, and the stem need not be a real word. Increases **recall** (more matches) but may reduce **precision** by wrong conflation.

Aspect	Stemming	Lemmatization
Method	Rule-based suffix stripping	Dictionary + part-of-speech analysis (morphology)
Output	May not be a word ('relev', 'queri')	Always a valid word ('relevant', 'query')
Speed	Very fast	Slower
Example	better -> better, studies -> studi	better -> good, studies -> study
Tools	PorterStemmer, LancasterStemmer, SnowballStemmer	WordNetLemmatizer, spaCy

Who invented the Porter stemmer?

Martin Porter (University of Cambridge) published 'An algorithm for suffix stripping' in 1980. It applies five phases of rewrite rules such as '-sses -> -ss', '-ies -> -i', '-ational -> -ate', and removes '-ing', '-ed', '-ion' depending on a 'measure' (number of vowel-consonant sequences) of the remaining stem. Porter later created Snowball, a language for writing stemmers, and the improved

'Porter2' English stemmer. Alternatives: Lovins (1968, the first published stemmer, by Julie Beth Lovins) and Paice/Husk (Lancaster, more aggressive).

2.3 Code explanation - cell by cell

Cell 1 - Imports

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
import warnings
warnings.filterwarnings('ignore')
```

- `re` - Python's regular-expression module, used for cleaning.
- `nltk` - Natural Language Toolkit; `stopwords` gives stop-word lists, `PorterStemmer` the stemmer, `word_tokenize` the tokenizer.
- `warnings.filterwarnings('ignore')` hides warning messages to keep output clean.
- **Prerequisite:** the data packages must be downloaded once: `nltk.download('punkt')` (newer NLTK also needs `'punkt_tab'`) and `nltk.download('stopwords')`. Without them you get a LookupError.

Cell 2 - The sample text

A triple-quoted string holds a three-sentence paragraph about IR. Note it contains line breaks, a hyphen ('Pre-processing'), full stops and an exclamation mark - deliberately noisy data to demonstrate cleaning.

Cell 3 - Lowercase and remove punctuation / numbers

```
clean = re.sub(r"[^a-z\s]", "", text.lower())
```

- `text.lower()` converts everything to lowercase first (this is why the regex only needs a-z).
- `re.sub(pattern, replacement, string)` replaces every match of the pattern with the replacement (here an empty string, i.e. deletes).
- Pattern `[^a-z\s]`: the square brackets define a character class; ^ inside it means **NOT**; so it matches any character that is **not** a letter a-z and **not** whitespace (\s = space, tab, newline).
- Result: full stops, '!', and the hyphen disappear. **Observe:** 'Pre-processing' became 'preprocessing' (one word) because the hyphen was deleted, not replaced by a space. Newlines are kept, which is harmless because the tokenizer treats them as whitespace.
- Side effect: letters like 'e' with accent, digits and symbols are removed too - fine for English demo, bad for multilingual data.

Cell 4 - Tokenization

```
tokens = word_tokenize(clean)
```

Returns a Python list of 38 tokens: ['information', 'retrieval', 'is', 'the', 'process', ...]. Notice 'documents' appears twice and 'retrieval' appears twice - duplicates are kept, so term frequency information is preserved.

Cell 5 - Stop-word removal

```
stop_words = (stopwords.words("english"))
filtered = [w for w in tokens if w not in stop_words]
```

- `stopwords.words('english')` returns a list of about 179 English stop words.

- A **list comprehension** keeps only tokens that are not in the list. 38 tokens drop to **23**: removed were is, the, of, from, a, are, and, to, the, by, the, of, is, an, in (15 words).
- Note that 'according' stays because NLTK's list does not contain it - stop-word lists are conventions, not laws.
- **Improvement:** wrap the list in `set(...)`. Membership test in a list is O(n) per word; in a set it is O(1).

Cell 6 - Stemming

```
ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered]
```

Creates a stemmer object and applies `.stem()` to each remaining word. Output: ['inform', 'retriev', 'process', 'obtain', 'relev', 'document', 'larg', 'collect', 'search', 'engin', 'retriev', 'rank', 'document', 'accord', 'queri', 'enter', 'user', 'preprocess', 'text', 'import', 'step', 'retriev', 'system'].

Cell 7 - Word -> stem mapping

```
for w, s in zip(filtered, stemmed):
    print(w, "->", s)
```

`zip` pairs the two lists element by element so we can compare each original word with its stem.

2.4 Interpreting the result

Original word(s)	Stem	Comment
retrieval, retrieving	retriev	**Good conflation** - two forms share one index term, so a query for 'retrieving' finds 'retrieval'.
documents	document	Plural removed (-s rule).
queries	queri	Rule '-ies -> -i'. Not a dictionary word but consistent with 'query' -> 'queri'.
relevant	relev	Suffix '-ant' removed; shows that stems are not real words.
information	inform	**Over-stemming**: 'information' and 'inform' merge, though they differ in meaning slightly.
engines	engin	Plural and final 'e' removed.
large	larg	Final 'e' removed.

Stage	Token count	Result
Original text	-	Mixed case, punctuation, newlines
After cleaning	-	lowercase letters and spaces only
After tokenization	38	list of words
After stop-word removal	23	15 stop words dropped
After stemming	23	Same count, but word forms merged: 'retriev' x3, 'document' x2 -> 19 unique stems

Key point: Stemming does not reduce token count; it reduces the **vocabulary** (distinct terms). Here 23 tokens contain 19 distinct stems, so the index dictionary becomes smaller and recall improves.

2.5 Limitations and improvements

- Replace hyphen with a space (`re.sub(r'[^a-z\s]', ' ', ...)`) if you want 'pre' and 'processing' separate.

- Use lemmatization (WordNet) when you need real words; use Snowball for multiple languages.
- Handle negation, entities and numbers when the application needs them (e.g. product search).
- Use a set for stop words, and add domain-specific stop words (e.g. 'patient' in medical corpora).

2.6 Likely viva questions

- **Why is the same pre-processing applied to the query?** So that query terms and index terms are in the same normal form and can match.
- **Stemming vs lemmatization?** Rules vs dictionary; stems may be non-words; lemmas are valid words.
- **Which stemmer is used and who made it?** Porter stemmer, Martin Porter, 1980.
- **Does removing stop words always help?** No - it can break phrase queries ('to be or not to be').
- **What does [^a-z\s] mean?** Any character other than a lowercase letter or whitespace.

Part 3 - Lab IR-2: Inverted Index and Boolean Search

3.1 Theory

An **inverted index** is the central data structure of every search engine. A normal (forward) index maps *document -> words it contains*. The **inverted** index reverses this: *word -> list of documents that contain it*. This is exactly like the index at the back of a textbook (term -> page numbers). Without it, to answer a query you would have to scan every document (linear time); with it you jump directly to the documents containing the query word.

Components

- **Dictionary / vocabulary / lexicon** - the set of all distinct terms (the keys of the dictionary in the code).
- **Postings list** - for each term, the list of document IDs (docIDs) where it occurs (the values). Each entry is a **posting**.
- A richer index also stores **term frequency** (tf), **positions** of the term inside the document (for phrase queries), and field information (title, body).

Who invented it?

There is no single inventor. The idea is centuries old: the **concordance** (an alphabetical list of words with locations) - the first Bible concordance was compiled by Hugh of St-Cher and colleagues around 1230. Computer-based *inverted files* appeared in the 1950s-60s in early IR systems and were central to Gerard Salton's SMART system. Modern implementations include Apache Lucene (Doug Cutting, 1999), Solr and Elasticsearch.

Index construction at scale

- **Sort-based (BSBI) and SPIMI** - build partial indexes in memory-sized blocks, write to disk, and merge.
- **MapReduce** - used by Google to index huge crawls in parallel.
- **Compression** - postings are stored as gaps (differences between successive docIDs) with variable-byte or gamma codes, shrinking the index dramatically.
- **Skip pointers** speed up intersection of long postings lists.

Boolean query processing: intersection merge

For an AND query, fetch the postings lists of each term and intersect them. If both lists are sorted, a two-pointer **merge** works in O(m + n). For multi-term queries, process terms from the **shortest** list upward to keep intermediate results small.

```
retrieval    -> [1, 2, 3]
information  -> [1, 3]
AND          -> [1, 3]
```

3.2 Code explanation

Cell - Complete program, part by part

Indexing function

```
index = {}

def add_document(doc_id, text):
    tokens = text.lower().split()
    for token in set(tokens):
        if token not in index:
            index[token] = []
        index[token].append(doc_id)
```

- `index = {}` - an empty Python dictionary: the inverted index. Key = term, value = list of docIDs.
- `text.lower().split()` - lowercase, then split on whitespace to obtain tokens. (A very simple tokenizer; no punctuation or stop-word handling.)
- `set(tokens)` - removes duplicate words in the *same* document, so a docID is added only once per term even if the word occurs many times. **Consequence:** term frequency is lost.
- `if token not in index` - first time we see this term, create an empty postings list.
- `index[token].append(doc_id)` - add the document to this term's postings list. Because documents are added in increasing docID order, each list stays sorted automatically.
- Note: iteration order of a set is arbitrary, so the order in which terms appear in the printed index can differ between runs; the content does not change.

Search function (Boolean AND)

```
def search(query):
    tokens = query.lower().split()
    if not tokens:
        return []
    result_docs = set(index.get(tokens[0], []))
    for token in tokens[1:]:
        result_docs = result_docs.intersection(set(index.get(token, [])))
    return sorted(result_docs)
```

- Query is normalised the same way as documents (lowercase + split).
- Empty query returns an empty list (guard clause).
- `index.get(tokens[0], [])` - safe lookup; returns an empty list if the term is not in the index instead of raising KeyError. Converted to a set to start the result.
- Loop over remaining terms and intersect: a document stays only if it contains **every** query term. This implements AND semantics.
- `sorted()` converts the set back to an ordered list of docIDs.

Main program - the three-document collection

```
add_document(1, "information retrieval is fun")
add_document(2, "retrieval of documents using inverted files")
add_document(3, "information systems and data retrieval")
```

DocID	Text	Distinct terms
1	information retrieval is fun	information, retrieval, is, fun
2	retrieval of documents using inverted files	retrieval, of, documents, using, inverted, files
3	information systems and data retrieval	information, systems, and, data, retrieval

The resulting inverted index (from the output)

Term	Postings list	Meaning
fun, is	[1]	only in document 1
information	[1, 3]	documents 1 and 3
retrieval	[1, 2, 3]	appears in every document
documents, of, inverted, files, using	[2]	only in document 2

Term	Postings list	Meaning
data, systems, and	[3]	only in document 3

Query trace

Query 'retrieval' -> postings [1, 2, 3] -> output [1, 2, 3].

Query 'information retrieval systems' step by step:

```
start : set(index['information']) = {1, 3}
AND retrieval : {1, 3} intersect {1, 2, 3} = {1, 3}
AND systems   : {1, 3} intersect {3}       = {3}
result = [3]    (only document 3 contains all three words)
```

3.3 Complexity and limitations

- Index build: O(total number of tokens). Search: O(sum of postings-list lengths); with sorted-list merge O(m + n) instead of Python sets' hashing.
- Only AND is supported - no OR, NOT, or phrase queries.
- No ranking - the result is an unordered set (Boolean model).
- No punctuation handling: 'fun.' would not match 'fun'. Fix: apply the IR-1 pre-processing (clean, stop words, stemming) before indexing *and* before searching.
- No term frequency or positions, so no TF-IDF scoring or phrase search.
- In-memory dictionary; real engines store the index on disk with compression.

3.4 Improved version (for extra marks)

Positional inverted index with AND / OR / NOT / phrase

```
from collections import defaultdict

index = defaultdict(dict)            # term -> {doc_id: [positions]}

def add_document(doc_id, text):
    for pos, tok in enumerate(text.lower().split()):
        index[tok].setdefault(doc_id, []).append(pos)

def search_and(q):
    lists = [set(index.get(t, {})) for t in q.lower().split()]
    return sorted(set.intersection(*lists)) if lists else []

def search_or(q):
    return sorted(set().union(*[set(index.get(t, {})) for t in q.lower().split()]))

def search_not(a, b):                # docs with a but not b
    return sorted(set(index.get(a, {})) - set(index.get(b, {})))

def phrase(q):                       # adjacent words, uses positions
    t = q.lower().split(); res = []
    for d in set.intersection(*[set(index.get(w, {})) for w in t]):
        for p in index[t[0]][d]:
            if all(p + i in index[t[i]][d] for i in range(1, len(t))):
                res.append(d); break
    return sorted(res)
```

Here term frequency is simply `len(index[term][doc])`, which opens the door to TF-IDF ranking.

3.5 Likely viva questions

- **What is an inverted index?** Mapping from terms to the documents containing them.
- **Why 'inverted'?** It inverts the document -> words relation.
- **Why use set(tokens) in add_document?** To avoid listing the same docID twice for one term.

- **Which model does search() implement?** Boolean retrieval with AND.
- **How would you add ranking?** Store tf, compute TF-IDF or BM25 scores, sort by score.
- **What is a postings list / dictionary?** List of docIDs for a term / the set of all terms.

Part 4 - Lab IR-3: Bayesian Network (Probabilistic Reasoning)

4.1 Theory

Probability foundations

IR is uncertain by nature - we never know for sure whether a document is relevant. Probability theory gives a principled way to reason about this uncertainty. The key tool is **Bayes' theorem**:

```
P(A | B) = P(B | A) x P(A) / P(B)
posterior = likelihood x prior / evidence
```

What is a Bayesian Network (BN)?

A Bayesian Network (belief network) is a **directed acyclic graph (DAG)** in which each **node** is a random variable and each **edge** A -> B means 'A directly influences B' (A is a *parent* of B). Every node stores a **Conditional Probability Distribution (CPD / CPT)** giving P(node | parents). The whole network represents the joint distribution compactly:

```
P(X1, X2, ..., Xn) = product over i of  P( Xi | Parents(Xi) )
```

Without the network, a joint table for n binary variables needs 2^n numbers; the BN needs only small local tables. **Inventor: Judea Pearl** (UCLA) developed Bayesian networks in the early-to-mid 1980s (term coined 1985; book 'Probabilistic Reasoning in Intelligent Systems', 1988) and received the 2011 ACM Turing Award for it.

Connection with Information Retrieval

- **Inference-network model of IR** - Howard Turtle and W. Bruce Croft (1990-91), used in the INQUERY system: document nodes -> term nodes -> query nodes; the probability that a query is satisfied is computed by inference.
- **Probabilistic ranking** (Probability Ranking Principle, BM25) - same philosophy of estimating P(relevant | document, query).
- **Naive Bayes** - the simplest BN (one class parent, many word children); the classic algorithm for spam filtering and text categorisation.
- Also used for query expansion, user modelling and medical diagnosis (as in this lab).

Key concepts used in the code

- **Evidence** - variables whose values we have observed (Age = Old, Cholesterol = High).
- **Query variable** - the unknown we want the distribution of (HeartDisease).
- **Maximum Likelihood Estimation (MLE)** - learn CPDs from data by simple counting: P(x | parents) = count(x, parents) / count(parents).
- **Variable Elimination** - an exact inference algorithm that sums out (eliminates) irrelevant variables one by one.

4.2 Code explanation

Cell 1 - Imports

```
import pandas as pd
import numpy as np
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.parameter_estimator import DiscreteMLE
from pgmpy.inference import VariableElimination
```

- **pandas** - DataFrame (table) for the dataset; **numpy** - random number generation.
- **pgmpy** - 'Probabilistic Graphical Models in Python'. `DiscreteBayesianNetwork` is the model class for variables with finite categories (in older pgmpy versions it was called BayesianNetwork / BayesianModel).
- `DiscreteMLE` - maximum-likelihood parameter estimator; `VariableElimination` - exact inference engine.

Cell 2 - Creating a synthetic dataset

```
np.random.seed(42)
data = pd.DataFrame({
    'Age': np.random.choice(['Young', 'Middle', 'Old'], size=500),
    'Cholesterol': np.random.choice(['Normal', 'High'], size=500),
    'HeartDisease': np.random.choice(['Yes', 'No'], size=500, p=[0.4, 0.6])
})
```

- `seed(42)` fixes the random generator so results are reproducible.
- `np.random.choice(options, size=500)` draws 500 random samples. Age: Young/Middle/Old with equal probability (1/3 each); Cholesterol: Normal/High (1/2 each).
- HeartDisease: 'Yes' with probability 0.4 and 'No' with 0.6 (`p=[0.4, 0.6]`).
- **Very important observation:** each column is generated **independently**. HeartDisease does *not* depend on Age or Cholesterol in this data. It is a toy dataset created only to demonstrate the mechanics.

Cell 3 - Viewing the data

Displaying `data` shows 500 rows x 3 columns, each row a patient, e.g. row 0: Old, High, Yes.

Cell 4 - Building and training the network

```
model = DiscreteBayesianNetwork([
    ('Age', 'HeartDisease'),
    ('Cholesterol', 'HeartDisease')
])
model.fit(data, estimator=DiscreteMLE())
print("Model Check Validated:", model.check_model())
```

- The edge list defines the structure: **Age -> HeartDisease <- Cholesterol**. Three nodes, two edges. HeartDisease has two parents; Age and Cholesterol are root nodes. This is a 'collider' (v-structure).
- `model.fit(data, estimator=DiscreteMLE())` learns the CPDs by counting in the data: P(Age), P(Cholesterol), and P(HeartDisease | Age, Cholesterol).
- `check_model()` validates that the graph is acyclic, every node has a CPD, CPDs are consistent with the graph, and each probability column sums to 1. It prints **True**.

CPD learned	Size	How computed
P(Age)	3 values	count(Age = a) / 500
P(Cholesterol)	2 values	count(Chol = c) / 500
P(HeartDisease | Age, Chol)	2 x (3 x 2) = 12 numbers (6 parent combinations)	count(HD = h, a, c) / count(a, c)

Joint distribution implied by the network: **P(Age, Chol, HD) = P(Age) x P(Chol) x P(HD | Age, Chol)**.

Cell 5 - Inference

```
inference = VariableElimination(model)
result = inference.query(
    variables=['HeartDisease'],
    evidence={'Age': 'Old', 'Cholesterol': 'High'}
)
print(result)
```

This asks: **P(HeartDisease | Age = Old, Cholesterol = High)** - 'what is the chance a patient who is old and has high cholesterol has heart disease?'. Because both parents are observed, variable elimination simply reads the matching column of the CPD (no hidden variable needs to be summed out).

Output

```
+------------------+--------------------+
| HeartDisease     |   phi(HeartDisease) |
+==================+====================+
| HeartDisease(No)  |             0.5926 |
| HeartDisease(Yes) |             0.4074 |
+------------------+--------------------+
```

4.3 Interpreting the output

- P(No) = 0.5926 and P(Yes) = 0.4074; they sum to 1.
- These values are close to the generating probabilities 0.6 and 0.4. The estimate comes from the subset of about 80 of the 500 patients who are Old and High-cholesterol (500 / 6 = ~83); a fraction such as 33/81 = 0.4074 is the kind of count-ratio MLE produces. The small difference from 0.4 is **sampling noise**.
- **Conclusion:** because the data were random and independent, the evidence (old + high cholesterol) did **not** change the probability. On real medical data the Yes probability for this group would be much higher than the baseline.
- 'phi' is pgmpy's notation for a factor (a table of values over variables).

Key point: Always say that this lab is a **demonstration of the method**, not a medical result. To make it meaningful, use a real dataset (e.g. UCI Heart Disease) or generate HeartDisease so that it depends on Age and Cholesterol.

4.4 Limitations and improvements

- Zero counts: if some parent combination never occurs, MLE cannot estimate its CPD. Use `BayesianEstimator` with a prior (Laplace / Dirichlet smoothing).
- Structure was given manually; it can also be learned from data (Hill-Climb search with BIC/K2 score, PC algorithm).
- Variable Elimination is exact but exponential in the worst case; for big networks use approximate methods (sampling, belief propagation).
- Only discrete variables here; continuous variables need Gaussian or discretised CPDs.

4.5 Likely viva questions

- **What is a Bayesian network?** DAG of random variables with conditional probability tables.
- **Who invented it?** Judea Pearl.

- **How is it related to IR?** Inference network model (Turtle and Croft), probabilistic ranking, Naive Bayes classification.
- **What does MLE do?** Estimates probabilities by relative frequency in the data.
- **What is variable elimination?** Exact inference by summing out variables in a chosen order.
- **Why is the answer near 0.4?** Because the synthetic data has no real dependency.

Part 5 - Lab IR-4: PageRank (Link Analysis)

5.1 Theory

Classic IR ranks by *content* (how often the query words occur). On the web, content alone is easy to spam, and millions of pages match any query. **Link analysis** uses the structure of the web: a hyperlink from page A to page B is a **vote** by A for B. **PageRank** is the algorithm that turns these votes into an importance score for every page.

Who invented it?

Larry Page and **Sergey Brin**, PhD students at Stanford University, developed PageRank in 1996-98 (with Rajeev Motwani and Terry Winograd as co-authors on the technical report 'The PageRank Citation Ranking: Bringing Order to the Web'). It is named after Larry Page (and 'web page'). The ideas come from **citation analysis** in bibliometrics (Eugene Garfield's impact factor): an academic paper cited by many important papers is important. The method became the core of Google, founded in 1998; the Stanford patent was licensed to Google. Independently, Jon Kleinberg published the HITS algorithm in 1998.

Random surfer model (intuition)

Imagine a person browsing the web at random. At each step, with probability **d** (damping factor, usually 0.85) the surfer clicks one of the outgoing links on the current page chosen uniformly; with probability **1 - d** (0.15) the surfer gets bored and jumps to a random page of the whole web. The PageRank of a page is the long-run fraction of time the surfer spends on it - the stationary distribution of this Markov chain.

Formula

```
PR(p) = (1 - d) / N  +  d x SUM over pages q that link to p of  PR(q) / L(q)

N    = total number of pages          d = damping factor (0.85)
L(q) = number of outgoing links of page q
The scores of all pages sum to 1.
```

- **A page is important if important pages link to it** (recursive definition).
- A page splits its score equally among its outgoing links (division by L(q)).
- The term (1 - d)/N is the 'teleportation' share every page receives, which also guarantees convergence, and handles spider traps.
- **Dangling nodes** (pages with no out-links) would leak score; implementations distribute their score uniformly over all pages (NetworkX does this).
- Computed by **power iteration**: start with equal scores 1/N and apply the formula repeatedly until values stop changing (tolerance), usually 50-100 iterations. Equivalent to finding the principal eigenvector of the Google matrix.

5.2 Code explanation

Cell 1-2 - Import and create graph

```
import networkx as nx
G = nx.DiGraph()
```

NetworkX is a Python library for graphs. `DiGraph` is a **directed** graph - edges have direction, which is required since links are one-way (A links to B does not mean B links to A).

Cell 3 - Add links (edges)

```
G.add_edges_from([('A','B'), ('A','C'), ('B','C'), ('C','A'), ('D','C')])
```

Nodes are created automatically from the edges. The web of 4 pages:

Page	Out-links (L)	In-links (who links to it)	In-degree
A	B, C (L = 2)	C	1
B	C (L = 1)	A	1
C	A (L = 1)	A, B, D	3
D	C (L = 1)	none	0

Cell 4 - Compute PageRank

```
pagerank = nx.pagerank(G, alpha=0.85)
```

`alpha` is the damping factor d. Returns a dictionary {page: score}. Defaults: max_iter = 100, tol = 1e-6 (recent NetworkX versions use SciPy internally). No dangling nodes exist in this graph because every page has at least one out-link.

Cell 5-6 - Display and rank

```
for page, rank in pagerank.items():
    print(page, ":", round(rank, 4))

for page, rank in sorted(pagerank.items(), key=lambda x: x[1], reverse=True):
    print(page, ":", round(rank, 4))
```

- `.items()` iterates over (key, value) pairs; `round(rank, 4)` shows 4 decimals.
- `sorted(..., key=lambda x: x[1], reverse=True)` sorts pairs by the score (second element) in descending order - the final ranking.

Output

```
A : 0.3725     B : 0.1958     C : 0.3942     D : 0.0375

Ranking:  C (0.3942) > A (0.3725) > B (0.1958) > D (0.0375)
```

5.3 Manual verification of the answer

With N = 4: (1 - d)/N = 0.15/4 = **0.0375**. Equations:

```
PR(A) = 0.0375 + 0.85 x PR(C)
PR(B) = 0.0375 + 0.85 x PR(A)/2
PR(C) = 0.0375 + 0.85 x ( PR(A)/2 + PR(B) + PR(D) )
PR(D) = 0.0375                        (no in-links, only teleportation share)
```

Check with the output: PR(A) = 0.0375 + 0.85(0.3942) = 0.3726 (matches 0.3725 up to rounding). PR(B) = 0.0375 + 0.85(0.1863) = 0.1958. PR(C) = 0.0375 + 0.85(0.1863 + 0.1958 + 0.0375) = 0.0375 + 0.85(0.4196) = 0.3942. The total 0.3725 + 0.1958 + 0.3942 + 0.0375 = 1.0000.

Power iteration - first two rounds

Iteration	PR(A)	PR(B)	PR(C)	PR(D)	Sum
0 (start)	0.2500	0.2500	0.2500	0.2500	1.0
1	0.2500	0.1438	0.5688	0.0375	1.0
2	0.5209	0.1438	0.2978	0.0375	1.0
... converge	0.3725	0.1958	0.3942	0.0375	1.0

Scores oscillate at first (C is big, then A gets C's vote, and so on) and settle after a few dozen rounds because the damping factor shrinks each swing by 0.85.

5.4 Interpretation

- **C is highest**: three pages (A, B, D) link to it, including A, which is itself important.
- **A is second**: it has only one in-link, but that link comes from C, and C gives *all* of its score to A (C has just one out-link). Quality of the linker matters, not only count.
- **B is third**: it receives only half of A's score (A splits its vote between B and C).
- **D is last**: nobody links to it, so it keeps only the minimum teleportation score 0.0375 = (1 - d)/N - this is the lowest possible PageRank in this setting.

5.5 Limitations and extensions

- Query-independent: PageRank is a global importance score; search engines combine it with hundreds of other signals (content relevance, freshness, user behaviour).
- Link spam and link farms try to cheat it; countered by TrustRank, nofollow tags and spam detection.
- Topic-sensitive / Personalised PageRank (random jump to a favourite set of pages), HITS (hubs and authorities), SALSA.
- PageRank is also used outside the web: citation networks, social networks, protein interaction networks, recommender systems and keyword extraction (TextRank).

5.6 Likely viva questions

- **What is PageRank and who made it?** Link-based importance score; Larry Page and Sergey Brin, Stanford, 1998.
- **What is the damping factor?** Probability 0.85 that the surfer follows a link; 0.15 for random jump.
- **Why is it needed?** To avoid dead ends and spider traps and to guarantee a unique convergent solution.
- **Why does D get 0.0375?** No incoming links - only the random-jump share (1 - 0.85)/4.
- **How is PageRank computed?** Power iteration until convergence.

Part 6 - Lab IR-5: Web Scraping and Crawling

6.1 Theory

Before anything can be indexed, the documents must be collected. On the web this is done by **crawlers** (also called spiders or bots) and **scrapers**.

	Web crawler	Web scraper
Goal	Discover and download many pages by following links	Extract specific data fields from known pages
Output	Copies of pages for indexing	Structured data (CSV, database)
Example	Googlebot	This lab: book name, price, links from one page

How a crawler works

- Start with **seed URLs** and put them in a **frontier** (queue of URLs to visit).
- Take a URL from the frontier, check **robots.txt** (the site's rules), **fetch** the page with HTTP.
- **Parse** the HTML, store content, **extract links**, convert relative links to absolute links, remove already-visited URLs (duplicate detection), add new ones to the frontier.
- Repeat. Breadth-first order is common; focused crawlers prioritise relevant topics; politeness rules limit requests per host (e.g. one request per second).
- Challenges: scale, freshness (re-crawling changing pages), duplicates, spider traps (infinite calendar pages), JavaScript-generated content (needs Selenium/Playwright), and legal/ethical limits.

History

Matthew Gray built the 'World Wide Web Wanderer' in 1993 (first web crawler, used to measure the size of the web). **Brian Pinkerton** created **WebCrawler** in 1994, the first search engine to index the full text of pages. Later Lycos, AltaVista, and Google's Googlebot followed. (Archie by Alan Emtage in 1990 preceded them, but it indexed FTP file names, not web pages.)

HTTP and HTML basics

- **HTTP GET** request asks the server for a page; the response has a **status code**: 200 OK, 301 redirect, 403 forbidden, 404 not found, 429 too many requests, 500 server error.
- **HTML** is a tree of tags: `<article class='product_pod'>` contains `<h3><a href=... title=...>`, `<p class='price_color'>`, `<img src=...>`. A parser turns this text into a searchable tree (the DOM).
- **Relative vs absolute URL**: 'catalogue/page.html' is relative; 'https://books.toscrape.com/catalogue/page.html' is absolute.

6.2 Code explanation

Cell 1 - Imports

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin
import pandas as pd
```

- **requests** - sends HTTP requests (GET) and returns a response object.
- **BeautifulSoup (bs4)** - parses HTML and offers methods like `find` and `find_all` to search the tree.

- **urljoin** - combines a base URL with a relative link to give an absolute URL.
- **pandas** - stores the extracted records in a DataFrame and writes CSV.

Cell 2-3 - Target site and request

```
url = "https://books.toscrape.com/"
response = requests.get(url)
product_data = []
```

books.toscrape.com is a sandbox website made specifically for practising scraping (a fake online bookshop) - legal and safe to scrape. `requests.get(url)` downloads the page. `product_data` is an empty list that will collect one dictionary per book.

Cell 4 - Parsing and extraction

```
if response.status_code == 200:
    soup = BeautifulSoup(response.text, "html.parser")
    products = soup.find_all("article", class_="product_pod")
    for product in products:
        name = product.h3.a["title"]
        price = product.find("p", class_="price_color").text
        link = product.h3.a["href"]
        product_link = urljoin(url, link)
        image = product.find("div", class_="image_container").img["src"]
        image_link = urljoin(response.url, image)
        product_data.append({"Product Name": name, "Price": price,
                             "Product Link": product_link, "Image Link": image_link})
else:
    print("Failed to access website:", response.status_code)
```

- **status_code == 200** - proceed only if the request succeeded; otherwise print the error (basic error handling).
- `BeautifulSoup(response.text, 'html.parser')` - build the parse tree using Python's built-in HTML parser (alternatives: lxml, html5lib).
- `find_all('article', class_='product_pod')` - returns every book card on the page. `class_` has an underscore because `class` is a reserved Python word. The page lists **20 books**.
- `product.h3.a['title']` - shortcut: first <h3> inside the card, its first <a>, then the attribute **title**. The *title attribute* is used instead of the link text because the visible text is truncated with '...' for long names, whereas the attribute holds the full name.
- `find('p', class_='price_color').text` - finds the price paragraph and takes its text (e.g. '£51.77').
- `h3.a['href']` is a **relative** link; `urljoin(url, link)` makes it absolute: https://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html.
- The image: locate `div.image_container`, take its `img` tag's `src` attribute (relative), and again make absolute with `urljoin(response.url, image)`. `response.url` is the final URL after redirects - slightly more robust than the original url.
- Each book is stored as a dictionary and appended to the list.

Cell 5 - Save to CSV

```
df = pd.DataFrame(product_data)
df.to_csv("products.csv", index=False, encoding="utf-8-sig")
```

A list of dictionaries becomes a table with columns Product Name, Price, Product Link, Image Link. `index=False` avoids writing the row numbers. `utf-8-sig` writes a byte-order mark so that Excel opens the file correctly with special characters.

Cell 6 - Display

`df` shows the 20 rows (A Light in the Attic £51.77, Tipping the Velvet £53.74, Soumission £50.10, Sharp Objects £47.82, Sapiens £54.23 ...). Long names and URLs appear cut with '...' - that is only pandas' display truncation; the CSV holds the full text.

6.3 The 'Â£' problem in the output

The price column shows **Â£51.77** instead of **£51.77**. This is a **character-encoding** error (mojibake). The server sends UTF-8 bytes, where '£' is two bytes (C2 A3), but the response header does not declare a charset, so `requests` guesses ISO-8859-1 (Latin-1) and decodes the two bytes as two characters: 'Â' and '£'. Because the data are saved with the garbled symbol, the CSV also contains 'Â£'.

Solutions

```
# Fix 1: tell requests the right encoding before using .text
response.encoding = "utf-8"
soup = BeautifulSoup(response.text, "html.parser")

# Fix 2: pass raw bytes; BeautifulSoup detects the encoding itself
soup = BeautifulSoup(response.content, "html.parser")

# Fix 3: clean afterwards, and convert to a number
price = float(price.replace("Â", "").replace("£", ""))
```

6.4 Extended version: crawl all pages politely

Pagination crawler (50 pages, 1000 books)

```
import time, requests, pandas as pd
from bs4 import BeautifulSoup
from urllib.parse import urljoin

url = "https://books.toscrape.com/"
headers = {"User-Agent": "StudentIRLab/1.0"}
rows = []

while url:
    r = requests.get(url, headers=headers, timeout=10)
    r.raise_for_status()
    r.encoding = "utf-8"
    soup = BeautifulSoup(r.text, "html.parser")
    for p in soup.select("article.product_pod"):
        rows.append({"Name": p.h3.a["title"],
                     "Price": float(p.select_one("p.price_color").text.strip("£")),
                     "Link": urljoin(r.url, p.h3.a["href"])})
    nxt = soup.select_one("li.next a")          # 'next' pagination button
    url = urljoin(r.url, nxt["href"]) if nxt else None
    time.sleep(1)                                # politeness delay

pd.DataFrame(rows).to_csv("all_books.csv", index=False, encoding="utf-8-sig")
```

This is a tiny **crawler**: the 'next' link plays the role of the frontier. A real crawler would additionally keep a visited-set and respect robots.txt.

6.5 Ethics and legality

- Check `/robots.txt` and the site's terms of service before scraping; prefer official APIs when available.
- Be polite: delay between requests, identify your bot with a User-Agent, do not overload servers.
- Do not collect personal data without lawful basis (privacy laws such as GDPR and India's DPDP Act), and respect copyright of the content.
- Dynamic pages built with JavaScript need Selenium or Playwright; a plain `requests` call only sees the initial HTML.

6.6 Likely viva questions

- **Crawler vs scraper?** Crawler follows links to discover pages; scraper extracts data fields.
- **What does BeautifulSoup do?** Parses HTML into a tree and lets you search it.
- **Why urljoin?** Converts relative links to absolute URLs.
- **What is robots.txt?** A file giving crawling permissions for bots.
- **Why 'Â£' appears?** UTF-8 bytes decoded as Latin-1; set response.encoding = 'utf-8'.
- **How do you scrape all pages?** Follow the 'next' link in a loop (pagination).

Part 7 - Revision Sheet

7.1 The five labs at a glance

Lab	Concept	IR pipeline stage	Key function / idea	Output summary
IR-1	Pre-processing	Text normalisation	re.sub, word_tokenize, stopwords, PorterStemmer	38 tokens -> 23 after stop words -> stems like 'retriev'
IR-2	Inverted index	Indexing and Boolean search	dict of term -> postings list; set intersection	'information retrieval systems' -> [3]
IR-3	Bayesian network	Probabilistic ranking / reasoning	DiscreteBayesianNetwork, DiscreteMLE, VariableElimination	P(Yes) = 0.4074, P(No) = 0.5926
IR-4	PageRank	Link-based ranking	nx.pagerank(G, alpha=0.85)	C 0.3942 > A 0.3725 > B 0.1958 > D 0.0375
IR-5	Web scraping	Crawling / acquisition	requests, BeautifulSoup, urljoin, pandas	20 books saved to products.csv

7.2 Important formulas

```
TF-IDF      w(t,d) = tf(t,d) x log(N / df(t))
Cosine      cos(q,d) = (q . d) / (|q| |d|)
BM25        sum IDF(t) x tf (k1+1) / (tf + k1 (1 - b + b |d|/avgdl))
Precision   = relevant retrieved / retrieved
Recall      = relevant retrieved / relevant total
F1          = 2PR / (P + R)
Bayes       P(A|B) = P(B|A) P(A) / P(B)
BN joint    P(X1..Xn) = product P(Xi | Parents(Xi))
MLE         P(x | pa) = count(x, pa) / count(pa)
PageRank    PR(p) = (1-d)/N + d x SUM_q PR(q)/L(q),   d = 0.85
```

7.3 Who did what - quick table

Invention	Inventor
Term 'Information Retrieval' (1950)	Calvin Mooers
Memex vision (1945)	Vannevar Bush
Automatic indexing, word-frequency idea	Hans Peter Luhn
Precision / recall, Cranfield tests	Cyril Cleverdon
SMART system, Vector Space Model, TF-IDF, relevance feedback	Gerard Salton
IDF	Karen Sparck Jones
BM25 / probabilistic relevance framework	Stephen Robertson and Karen Sparck Jones
Porter stemmer (1980)	Martin Porter
Bayesian networks (1980s)	Judea Pearl
Inference network for IR (1990)	Howard Turtle and W. Bruce Croft
First web crawler (1993) / WebCrawler (1994)	Matthew Gray / Brian Pinkerton
PageRank and Google (1998)	Larry Page and Sergey Brin
HITS (1998)	Jon Kleinberg
Lucene (1999)	Doug Cutting

7.4 Glossary

Term	Meaning
Corpus	The collection of documents being searched.
Token / term	A single word unit after tokenization / a normalised index word.
Stop word	Very common word with little meaning, removed from the index.
Stemming	Cutting word endings to get a root form.
Posting	One entry (docID, maybe tf/positions) in a postings list.
Query	The user's expression of an information need.
Relevance	How well a document satisfies the information need.
Precision / Recall	Accuracy / completeness of retrieved results.
CPD	Conditional probability distribution P(node | parents).
Evidence	Observed values of variables in probabilistic inference.
Damping factor	Probability (0.85) of following a link in PageRank.
Dangling node	A page with no outgoing links.
Crawler / spider	Program that automatically fetches pages by following links.
robots.txt	File that tells crawlers which parts of a site they may visit.
Frontier	Queue of URLs still to be crawled.

7.5 How the labs fit together

A mini search engine would work like this: **IR-5** crawls pages -> **IR-1** cleans each page's text into terms -> **IR-2** builds the inverted index -> a query is cleaned with IR-1 and matched through the index -> candidate documents are ranked by content score (TF-IDF/BM25, or a probabilistic model as in **IR-3**) combined with link importance from **IR-4** (PageRank). Good luck with your exam and practicals!